

EMBEDDED SYSTEMS LAB REPORT

Automatic Street Noise Monitoring System

Submitted by

Susmita Sain

Roll No: 2301221

Lab Partner

Tanushka Sharma

Roll No: 2301227

Course: Embedded Systems Laboratory

Instructor: Dr. Sheikh Wasim Hussain



Department of Electronics and Communication Engineering
Indian Institute of Information Technology

April 20, 2026

1 Aim

To develop a system that measures street noise levels using a microphone and ESP32, and displays the data in real time on a web interface via Raspberry Pi.

2 Abstract

This project presents an Automatic Street Noise Monitoring System using ESP32, a microphone sensor, and Raspberry Pi. The system captures environmental sound, converts it into decibel values, and transmits the data wirelessly. The Raspberry Pi logs the data and hosts a web-based interface to display real-time noise levels and alerts. This system can be used for monitoring noise pollution in urban environments.

3 System Design and Flowchart

3.1 Equipment Used:

- MAX9814 Microphone Sensor
- ESP32 Microcontroller
- Raspberry Pi
- WiFi
- Web-based Interface

3.2 System Level Design

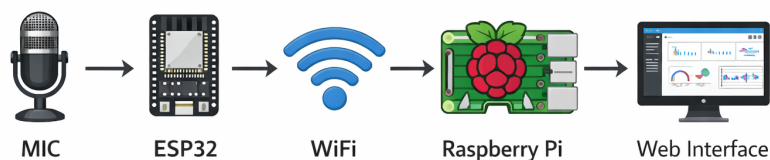


Figure 1: Flowchart

The system begins by capturing environmental sound using a microphone sensor. The analog signal generated by the microphone is then fed into the ESP32, where it is converted into a digital signal using the built-in ADC. The ESP32 processes this signal to calculate the noise level in decibels (dB).

Once the noise level is computed, the ESP32 transmits the data to the Raspberry Pi over a WiFi network using HTTP protocol. The Raspberry Pi acts as a server, receiving and storing the incoming data. Finally, the processed data is displayed on a web-based interface, allowing real-time monitoring of noise levels.

4 Sensors and Subsystems

Microphone (MAX9814): Captures sound signals and provides analog output. It has an inbuilt amplifier with automatic gain control (AGC) for stable measurements.

ESP32: Reads analog data, calculates noise levels in dB, and sends data via WiFi. It has a built-in ADC and wireless capability.

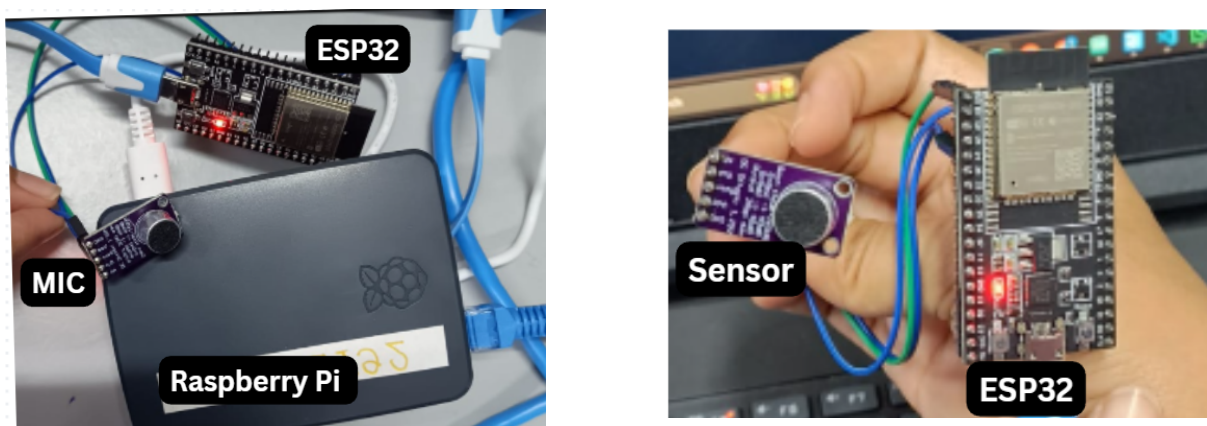


Figure 2: Subsystems Overview

Raspberry Pi: Acts as a server to receive and store data, and hosts the web interface.

Web Interface: Displays real-time noise levels and basic alerts for easy monitoring.

5 Schematic Explanation

The schematic shows the overall working of the Automatic Street Noise Monitoring System.

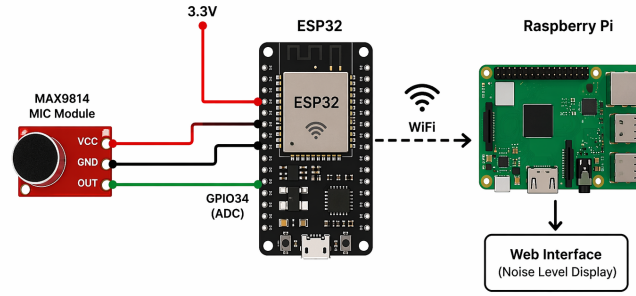


Figure 3: Schematic

The MAX9814 microphone captures sound and provides an analog output to the ESP32 through GPIO34 (ADC pin). The ESP32 processes this signal and calculates the noise level in decibels (dB).

Connections:

- MIC VCC → ESP32 3.3V
- MIC GND → ESP32 GND
- MIC OUT → ESP32 GPIO34

The ESP32 then sends the data to the Raspberry Pi over WiFi using HTTP. The Raspberry Pi acts as a server, storing the data and hosting a web interface. The web page displays the noise level in real time for easy monitoring.

6 Code Implementation

6.1 ESP32 Code

```

1 #include <WiFi.h>           // Library for WiFi
2 #include <HTTPClient.h>     // Library for HTTP requests
3 #include <math.h>           // For logarithmic calculations
4
5 const char* ssid = "SCAL";   // WiFi SSID
6 const char* password = "scal005@iiitg"; // WiFi Password
7
8 // Raspberry Pi server address
9 const char* serverName = "http://192.168.31.137:5000/data";
10
11 #define MIC_PIN 34          // Microphone connected to GPIO34 (ADC pin)
12
13 // Setup function runs once
14 void setup() {

```

```

15 Serial.begin(115200); // Start serial communication
16
17 // Connect to WiFi
18 WiFi.begin(ssid, password);
19 Serial.print("Connecting to WiFi");
20
21 // Wait until WiFi connects
22 while (WiFi.status() != WL_CONNECTED) {
23     delay(500);
24     Serial.print(".");
25 }
26
27 Serial.println("\nWiFi Connected!");
28 Serial.print("ESP32 IP: ");
29 Serial.println(WiFi.localIP()); // Print ESP32 IP
30 }
31
32 // Loop runs continuously
33 void loop() {
34
35     int signalMax = 0; // Maximum signal value
36     int signalMin = 4095; // Minimum signal value
37
38     unsigned long startMillis = millis(); // Start time
39
40     // Collect samples for 50 ms
41     while (millis() - startMillis < 50) {
42         int sample = analogRead(MIC_PIN); // Read analog value
43
44         if (sample > signalMax) signalMax = sample;
45         if (sample < signalMin) signalMin = sample;
46     }
47
48     float peakToPeak = signalMax - signalMin; // Amplitude
49     float voltage = (peakToPeak * 3.3) / 4095; // Convert to
        voltage
50     float dB = 20 * log10(voltage); // Convert to dB
51
52     dB = dB * 2; // Calibration factor
53
54     // Print on Serial Monitor

```

```

55 Serial.print("Noise Level: ");
56 Serial.print(dB);
57 Serial.println(" dB");
58
59 // Send data to Raspberry Pi
60 if (WiFi.status() == WL_CONNECTED) {
61
62     HTTPClient http;
63
64     http.begin(serverName); // Connect to server
65     http.addHeader("Content-Type", "application/json");
66
67     // Create JSON data
68     String jsonData = "{\"noise\":\"" + String(dB) + "\"}";
69
70     int responseCode = http.POST(jsonData); // Send POST request
71
72     Serial.print("Sent to server | Response: ");
73     Serial.println(responseCode);
74
75     http.end(); // Close connection
76
77 } else {
78     Serial.println("WiFi Disconnected!");
79 }
80
81 delay(2000); // Send data every 2 seconds
82 }

```

6.2 Web Interface (HTML Code)

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Noise Monitor</title>
5
6 <style>
7 /* Page background and font */
8 body {
9     margin: 0;
10    font-family: Arial, sans-serif;

```

```

11     background: #0f172a;
12     color: white;
13     text-align: center;
14 }
15
16 /* Main heading */
17 .header {
18     padding: 20px;
19     font-size: 26px;
20     font-weight: bold;
21     background: #020617;
22 }
23
24 /* Center content */
25 .container {
26     display: flex;
27     justify-content: center;
28     align-items: center;
29     height: calc(100vh - 80px);
30 }
31
32 /* Card design */
33 .card {
34     background: #1e293b;
35     padding: 30px;
36     border-radius: 15px;
37     width: 280px;
38     box-shadow: 0 0 20px rgba(0,0,0,0.4);
39 }
40
41 /* Title inside card */
42 h1 {
43     color: #38bdf8;
44 }
45
46 /* dB value */
47 #db {
48     font-size: 55px;
49     margin: 20px 0;
50 }
51

```

```

52 /* Status box */
53 #status {
54     font-size: 18px;
55     padding: 6px;
56     border-radius: 8px;
57 }
58 </style>
59
60 </head>
61
62 <body>
63
64 <!-- Main heading -->
65 <div class="header">
66     Automatic Street Noise Monitoring System
67 </div>
68
69 <div class="container">
70     <div class="card">
71         <h1>Noise Monitor</h1>
72
73         <!-- Noise value display -->
74         <div id="db">-- dB</div>
75
76         <!-- Noise status -->
77         <div id="status">Waiting...</div>
78     </div>
79 </div>
80
81 <script>
82
83 // URL to fetch data from Flask server
84 const URL = "/get-data";
85
86 // Function to classify noise level
87 function getLevel(db) {
88     if (db < 45) return ["QUIET", "#22c55e"];
89     if (db < 60) return ["NORMAL", "#3b82f6"];
90     if (db < 75) return ["LOUD", "#eab308"];
91     return ["ALERT", "#ef4444"];
92 }

```



```

93
94 // Update UI with new data
95 function updateUI(db) {
96     let level = getLevel(db);
97
98     // Show dB value
99     document.getElementById("db").innerText = db.toFixed(1) + " dB"
100     ;
101
102     // Show status
103     let status = document.getElementById("status");
104     status.innerText = level[0];
105     status.style.background = level[1];
106     status.style.color = "black";
107 }
108
109 // Fetch data from server
110 function fetchData() {
111     fetch(URL)
112     .then(res => res.json())
113     .then(data => {
114         if (data.noise !== undefined) {
115             updateUI(data.noise);
116         }
117     });
118 }
119
120 // Update every 2 seconds
121 setInterval(fetchData, 2000);
122
123 </script>
124
125 </body>
126 </html>

```

6.3 Flask Server Code

```

1 from flask import Flask, request, jsonify, render_template
2
3 app = Flask(__name__) # Create Flask app
4

```

```

5 latest_noise = 0 # Variable to store latest noise value
6
7 # Home route (loads webpage)
8 @app.route('/')
9 def home():
10     return render_template('index.html')
11
12 # Route to receive data from ESP32
13 @app.route('/data', methods=['POST'])
14 def data():
15     global latest_noise
16     data = request.json # Receive JSON data
17     latest_noise = data['noise'] # Store noise value
18
19     print("Noise Received:", latest_noise, "dB") # Print in
20         terminal
21
22     return {"status": "ok"} # Send response back
23
24 # Route to send data to webpage
25 @app.route('/get-data')
26 def get_data():
27     return jsonify({"noise": latest_noise})
28
29 # Run server
30 app.run(host='0.0.0.0', port=5000)

```

7 Implementation Steps

1. **Hardware Setup:** Connect the MAX9814 microphone to ESP32 (VCC to 3.3V, GND to GND, OUT to GPIO34 for analog input).
2. **Signal Reading:** ESP32 reads analog values from the microphone using its ADC over a short time interval.
3. **Signal Processing:** From the collected samples, maximum and minimum values are found to calculate the peak-to-peak signal.
4. **dB Calculation:** The signal is converted into voltage and then into decibel using:

$$dB = 20 \log_{10}(V)$$

5. **WiFi Connection:** ESP32 connects to a WiFi network using the given SSID and password.
6. **Data Transmission:** The calculated noise value is sent to the Raspberry Pi using an HTTP POST request in JSON format.
7. **Server Setup:** A Flask server on Raspberry Pi receives the data and stores the latest value.
8. **Web Interface:** A web page fetches the data from the server and displays the noise level.
9. **Real-Time Monitoring:** The webpage updates continuously to show live noise readings.

8 Results

The developed system was successfully tested for real-time noise monitoring. The ESP32 was able to capture sound signals, process them, and transmit the noise values to the Raspberry Pi at regular intervals.

The Raspberry Pi server correctly received and logged the data, and the web interface displayed the noise levels dynamically.

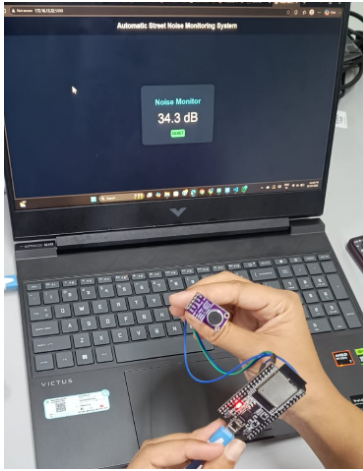


Figure 4: Hardware Setup

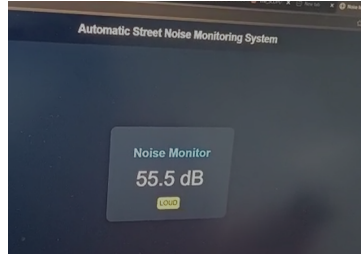


Figure 5: Web Interface

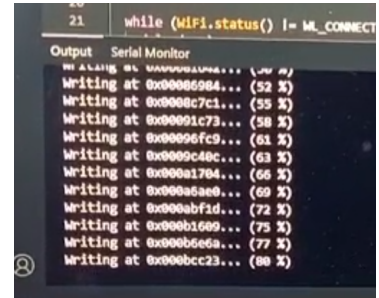


Figure 6: Terminal Output

The system updates noise values approximately every 2 seconds. It was observed that the noise levels vary depending on environmental conditions, demonstrating the effectiveness of the system in real-time monitoring.

9 Conclusion

The project successfully demonstrates a real-time noise monitoring system using ESP32 and Raspberry Pi. The system effectively captures sound, processes it, and displays it on a web interface. It is a low-cost and efficient solution for monitoring environmental noise.

10 Future Scope

- Integration with cloud platforms
- Mobile application development
- AI-based noise classification
- Multi-location monitoring system